Python 与机器学习学习笔记

梧桐

2026 年 10 月 13 日

目录

第一章 Git 与 GitHub **1**

1.1 Git 基础概念 1

1.1.1 什么是 Git 1

1.1.2 Git 与 GitHub 的区别 1

1.1.3 Git 的基本工作流程 1

1.2 Git 安装与配置 2

1.2.1 检查 Git 是否安装 2

1.2.2 配置用户名 2

1.2.3 配置邮箱 2

1.2.4 查看 Git 配置 3

1.2.5 设置默认分支 3

1.3 GitHub 仓库管理 3

1.3.1 创建 GitHub 仓库 3

1.3.2 切换工作目录 4

1.3.3 克隆远程仓库 4

1.3.4 使用临时代理克隆 4

1.3.5 进入本地仓库 5

1.3.6 查看远程仓库地址 5

1.4 Git 本地版本管理 5

1.4.1 查看仓库状态 5

1.4.2 添加指定文件到暂存区 5

1.4.3 添加当前目录的全部修改 6

1.4.4 取消暂存 6

1.4.5 查看未暂存修改 6

1.4.6 查看已暂存修改 6

1.5 Git Commit：创建本地提交 6

1.5.1 创建提交 6

1.5.2 查看提交历史 7

1.5.3 简洁显示提交历史 7

1.5.4 查看某次提交的详细内容 7
1.6 Git Push：上传代码 7
1.6.1 普通上传 7
1.6.2 使用临时代理上传 8
1.6.3 检查上传结果 8
1.7 Git Pull：获取远程更新 8
1.7.1 拉取远程更新 8
1.7.2 使用临时代理拉取 8
1.7.3 查看分支 8
1.8 Git 版本恢复 9
1.8.1 恢复未暂存的修改 9
1.8.2 撤销历史提交 9
1.9 Git 忽略文件 9
1.9.1 .gitignore 的作用 9
1.10 Git 完整操作实例 10
1.11 Git 常用命令速查 12
1.12 Git 分支管理 12
1.12.1 分支的作用 12
1.12.2 创建并切换分支 12
1.12.3 在功能分支提交代码 12
1.12.4 合并并推送主分支 13
1.13 Pull Request 与代码审查 13
1.13.1 Pull Request 的作用 13
1.13.2 推送功能分支 13
1.13.3 GitHub 网页创建 PR 14
1.13.4 审查与合并 14
1.14 Jupyter Notebook 与科研项目管理 14
1.14.1 Python 脚本与 Notebook 的区别 14
1.14.2 科研仓库目录组织 14
1.14.3 Notebook 中的科研计算示例 14
1.14.4 提交 Notebook 前的检查 15
1.14.5 科研项目可复现性 15
1.15 本章总结 15

第二章 Markdown 与 LaTeX 17
2.1 Markdown 基础 17
2.2 LaTeX 基础 17

第三章 Python 编程基础 **18**
3.1 Python 开发环境 18
3.2 Python 基础语法 18
3.3 Python 数据类型 18
3.4 Python 控制流程 18
3.5 Python 函数 18

第一章　Git 与 GitHub

1.1　Git 基础概念

1.1.1　什么是 Git

Git 是一种分布式版本控制系统
（Distributed Version Control System），
主要用于记录文件的修改历史、管理不同版本，
以及支持多人协作开发。
在 Python 编程与机器学习研究中，Git 可以用于：

- 保存 Python 程序的历史版本；
- 比较不同算法实现之间的代码差异；
- 在程序修改出错时恢复稳定版本；
- 管理实验脚本、配置文件和研究代码；
- 与 GitHub、Gitee 等平台进行远程同步。

1.1.2　Git 与 GitHub 的区别

Git 是本地版本控制工具，
GitHub 是基于 Git 的远程代码托管平台。

工具	主要功能
Git	本地代码版本管理
GitHub	远程仓库托管与团队协作
Gitee	远程仓库托管与团队协作

1.1.3　Git 的基本工作流程

Git 的工作流程通常包含四个部分：

1. 工作区（Working Directory）；
2. 暂存区（Staging Area）；
3. 本地仓库（Local Repository）；
4. 远程仓库（Remote Repository）。

其核心命令为：

```
git add

git commit

git push
```

需要注意，`git commit` 仅创建本地提交，
并不意味着代码已经上传到 GitHub。

1.2 Git 安装与配置

1.2.1 检查 Git 是否安装

在 Git Bash 或 VS Code 终端输入：

```
git --version
```

作用：查看 Git 的安装版本。
如果终端能够正常显示版本号，
说明 Git 已安装并可以使用。

1.2.2 配置用户名

```
git config --global user.name "Your Name"
```

1.2.3 配置邮箱

```
git config --global user.email "your-email@example.com"
```

1.2.4 查看 Git 配置

查看用户名：

```
git config --global user.name
```

查看邮箱：

```
git config --global user.email
```

查看全部有效配置：

```
git config --list
```

1.2.5 设置默认分支

```
git config --global init.defaultBranch main
```

1.3 GitHub 仓库管理

1.3.1 创建 GitHub 仓库

在 GitHub 网站中选择 New Repository，
填写仓库名称及相关信息。
常见选项包括：

- Repository name：仓库名称；
- Description：仓库描述；
- Public：公开仓库；
- Private：私有仓库；
- README：项目说明文件。

1.3.2 切换工作目录

在 Git Bash 中切换到 D 盘指定目录：

```
cd /d/GitHub
```

查看当前工作目录：

```
pwd
```

查看目录中的文件：

```
ls
```

1.3.3 克隆远程仓库

命令格式：

```
git clone <repository-url>
```

实际示例：

```
git clone https://github.com/Zz-wutong/my-python-learning.git
```

作用：将远程仓库下载到本地，
并建立与 GitHub 仓库的关联。

1.3.4 使用临时代理克隆

如果 GitHub 网络连接失败，
且本机已经配置了 HTTP 代理，
可以使用：

```
git -c http.proxy=http://127.0.0.1:7897 clone https://github.com/Zz-wutong/my-
    python-learning.git
```

1.3.5 进入本地仓库

```
cd my-python-learning
```

1.3.6 查看远程仓库地址

```
git remote -v
```

1.4 Git 本地版本管理

1.4.1 查看仓库状态

```
git status
```

该命令用于检查当前分支、
已修改文件、暂存文件和未跟踪文件。
常见状态说明：

- `modified`：文件已被修改；
- `untracked`：文件尚未被 Git 跟踪；
- `Changes to be committed`：
  文件已经加入暂存区；
- `working tree clean`：
  工作区没有待提交修改。

1.4.2 添加指定文件到暂存区

```
git add code/hello.py
```

作用：将指定文件的当前修改加入暂存区。

1.4.3 添加当前目录的全部修改

```
git add .
```

作用：暂存当前目录及子目录中的新增、
修改和删除记录。
注意：
使用该命令前，应检查是否存在
不需要提交的配置文件或实验数据。

1.4.4 取消暂存

```
git restore --staged code/hello.py
```

作用：将指定文件从暂存区移出，
但保留工作区中的修改。

1.4.5 查看未暂存修改

```
git diff
```

作用：比较工作区与暂存区之间的差异。

1.4.6 查看已暂存修改

```
git diff --staged
```

作用：检查下一次提交将包含哪些修改。

1.5 Git Commit：创建本地提交

1.5.1 创建提交

```
git commit -m "Update Python example"
```

例如，新增试件体积计算代码后：

```
git commit -m "Add specimen volume calculation"
```

1.5.2 查看提交历史

```
git log
```

作用：查看提交时间、作者、Commit ID
及提交说明。

1.5.3 简洁显示提交历史

```
git log --oneline
```

1.5.4 查看某次提交的详细内容

```
git show <commit-id>
```

作用：查看指定提交引入的代码修改。

1.6 Git Push：上传代码

1.6.1 普通上传

```
git push origin main
```

其中：

- `push`：上传本地提交；
- `origin`：远程仓库名称；
- `main`：目标分支。

1.6.2 使用临时代理上传

```
git -c http.proxy=http://127.0.0.1:7897 push origin main
```

1.6.3 检查上传结果

上传完成后，可以登录 GitHub，
进入对应仓库检查文件内容和提交记录。
也可以再次执行：

```
git status
```

注意：工作区干净不一定代表
本地提交已经成功上传，
仍需确认远程分支同步状态。

1.7 Git Pull：获取远程更新

1.7.1 拉取远程更新

```
git pull origin main
```

作用：获取远程分支的新提交，
并尝试整合到当前本地分支。

1.7.2 使用临时代理拉取

```
git -c http.proxy=http://127.0.0.1:7897 pull origin main
```

1.7.3 查看分支

查看本地分支：

```
git branch
```

查看远程分支：

```
git branch -r
```

查看全部分支：

```
git branch -a
```

1.8 Git 版本恢复

1.8.1 恢复未暂存的修改

```
git restore code/hello.py
```

作用：将工作区中的文件恢复为暂存区版本。

警告：

该操作会丢弃文件中尚未暂存的修改，

使用前应确认不再需要这些内容。

1.8.2 撤销历史提交

```
git revert <commit-id>
```

作用：创建一个新的提交，

用于撤销指定提交带来的修改。

对于已经上传到 GitHub 的历史提交，

通常优先考虑使用 `git revert`。

1.9 Git 忽略文件

1.9.1 .gitignore 的作用

`.gitignore` 用于指定不需要 Git

跟踪的文件或文件夹。

例如，Python 项目可以忽略：

```
__pycache__/

*.pyc

.ipynb_checkpoints/

.venv/
```

如果不希望提交 VS Code 的本地配置，
还可以添加：

```
.vscode/
```

1.10 Git 完整操作实例

假设在 VS Code 中修改了 Python 文件：
`code/hello.py`
例如新增：

```
length = 400

width = 50

thickness = 0.5



volume = length * width * thickness



print("Specimen volume:", volume, "mm^3")
```

对应的 Git 操作如下：

```
# Step 1: Check status
```

```
git status



# Step 2: Stage changes

git add code/hello.py



# Step 3: Review staged changes

git diff --staged



# Step 4: Commit locally

git commit -m "Add volume calculation"



# Step 5: Push to GitHub

git push origin main
```

需要临时代理时：

```
git -c http.proxy=http://127.0.0.1:7897 push origin main
```

1.11 Git 常用命令速查

命令	功能
`git --version`	查看版本
`git status`	查看仓库状态
`git add`	暂存修改
`git commit`	创建本地提交
`git push`	上传提交
`git pull`	拉取远程更新
`git diff`	查看修改差异
`git log`	查看提交历史
`git restore`	恢复文件
`git revert`	撤销历史提交
`git remote -v`	查看远程地址
`git branch`	查看分支

1.12 Git 分支管理

1.12.1 分支的作用

分支（Branch）用于在不直接改变主分支的情况下开发新功能。`main` 通常保存稳定版本，`feature/...` 用于独立开发。

1.12.2 创建并切换分支

```
git branch
# Create and switch to a feature branch
git switch -c feature/mass-calculation
# Confirm current branch
git branch
```

1.12.3 在功能分支提交代码

例如为试件添加质量计算：

```
# density: g/cm^3, volume: mm^3
density = 6.45
mass = volume / 1000 * density
print("Specimen mass:", mass, "g")
```

```
git add code/hello.py
git commit -m "Add specimen mass calculation"
```

1.12.4 合并并推送主分支

```
git switch main
git merge feature/mass-calculation
git branch -d feature/mass-calculation
git -c http.proxy=http://127.0.0.1:7897 push origin main
```

若主分支在分支创建后没有新的独立提交，Git 通常可以执行快进合并（Fast-forward）。出现冲突时应检查冲突文件，手动解决并验证后再完成合并。

1.13 Pull Request 与代码审查

1.13.1 Pull Request 的作用

Pull Request（PR）是向目标分支提出合并修改的请求，提供代码差异查看、评论、检查与审查机制。PR 与 `git pull` 不是同一个操作。

1.13.2 推送功能分支

```
git switch main
git -c http.proxy=http://127.0.0.1:7897 pull --ff-only origin main
git switch -c feature/section-area
```

修改 `code/hello.py`:

```
# Rectangular cross-sectional area in mm^2
cross_sectional_area = width * thickness
print("Cross-sectional area:", cross_sectional_area, "mm^2")
```

```
git add code/hello.py
git commit -m "Add cross-sectional area calculation"
git -c http.proxy=http://127.0.0.1:7897 push -u origin feature/section-area
```

1.13.3 GitHub 网页创建 PR

进入仓库，点击 **Compare & pull request**，或依次点击 **Pull requests**、**New pull request**。选择 `base: main` 和 `compare: feature/section-area`，填写标题、描述后创建 PR。

1.13.4 审查与合并

在 **Files changed** 检查代码差异，确认修改范围、公式、单位和敏感信息。审查通过后点击 **Merge pull request**，再确认合并。网页合并后，需要同步本地：

```
git switch main
git -c http.proxy=http://127.0.0.1:7897 pull --ff-only origin main
git branch -d feature/section-area
```

1.14 Jupyter Notebook 与科研项目管理

1.14.1 Python 脚本与 Notebook 的区别

`.py` 适合可复用的算法与程序，`.ipynb` 适合交互式计算、Markdown 说明、公式与可视化。Notebook 文件还可能保存运行输出和执行计数，因此每次运行后 Git 差异可能较大。

1.14.2 科研仓库目录组织

```
my-python-learning/
  code/               # Python source code
  notebooks/          # Jupyter notebooks
  data/               # Small, shareable datasets
  results/            # Selected outputs
  README.md
  .gitignore
```

1.14.3 Notebook 中的科研计算示例

在 `notebooks/01_python_basics.ipynb` 中新建代码单元：

```
import numpy as np
import matplotlib.pyplot as plt
length, width = 400, 50
```

```
thicknesses = np.array([0.3, 0.4, 0.5, 0.6, 0.7])
volumes = length * width * thicknesses
plt.plot(thicknesses, volumes, "o-")
plt.xlabel("Thickness (mm)")
plt.ylabel("Volume (mm^3)")
plt.grid(True)
plt.show()
```

1.14.4 提交 Notebook 前的检查

建议清除不必要的运行输出，检查文件是否包含本地路径、令牌或敏感数据。只添加需要上传的 Notebook，避免直接执行 `git add .` 误提交其他文件。

```
git status
git add notebooks/01_python_basics.ipynb
git diff --staged --stat
git commit -m "Add first scientific computing notebook"
git -c http.proxy=http://127.0.0.1:7897 push origin main
```

1.14.5 科研项目可复现性

建议记录代码版本、数据来源、Python 环境与依赖库。大型数据和训练模型应根据大小与保密要求采用独立存储或 Git LFS，不应直接将涉密数据上传到 GitHub。

1.15 本章总结

Git 的核心操作流程为：

$$\texttt{git status} \longrightarrow \texttt{git add} \longrightarrow \texttt{git commit} \longrightarrow \texttt{git push}$$

本章介绍了 Git 的基础概念、
安装配置、仓库克隆、本地提交、
远程同步、版本恢复和文件忽略机制。

在后续 Python 与机器学习学习中，
可以利用 Git 管理算法实现、
实验脚本及配置文件，
并通过 GitHub 或 Gitee 保存和共享代码。
建议养成以下习惯：

1. 修改代码前检查仓库状态；
2. 提交前检查代码差异；
3. 每次提交使用明确的说明；
4. 定期同步远程仓库；
5. 不上传密码、访问令牌及敏感数据。

第二章　Markdown 与 LaTeX

2.1　Markdown 基础

Markdown 是一种轻量级标记语言。

2.2　LaTeX 基础

LaTeX 适合编写数学公式和学术文档。

第三章　Python 编程基础

3.1　Python 开发环境

主要使用 Anaconda、Jupyter Notebook 和 VS Code。

3.2　Python 基础语法

```
print("Hello, Python!")
```

3.3　Python 数据类型

3.4　Python 控制流程

3.5　Python 函数